Documentación de Avance: Pipeline Distribuido IoT/Edge

#laterceraeslavencida

1. Objetivo del Avance

El presente documento detalla el cumplimiento del primer hito del proyecto, demostrando la implementación de la estructura base del repositorio, la configuración de contenedores, la topología de red virtual, y el desarrollo inicial del prototipo en Rust. Se evidencia el trabajo colaborativo distribuido, garantizando que el diseño cumple con los requisitos de tolerancia a fallos, procesamiento perimetral (Edge) y centralización de telemetría.

2. Organización y Estructura del Repositorio

El proyecto cuenta con un repositorio centralizado gestionado de forma colaborativa, cumpliendo estrictamente con la estructura de directorios requerida para la separación de responsabilidades y la modularidad del código. En la raíz del repositorio se implementó un *Workspace* de Cargo (`Cargo.toml`) para administrar de manera unificada las dependencias compartidas de los microservicios, tales como `tokio`, `serde` y `serde_json`.

Estructura de Directorios:

- `/vpn/`: Contiene las plantillas de configuración de WireGuard sanitizadas (`wg0.conf.template`), omitiendo las llaves privadas reales por razones de seguridad.
- `/docker/`: Aloja el archivo global de orquestación `docker-compose.yml` y los subdirectorios con los `Dockerfile` específicos para cada nodo del sistema (`/sensor/`, `/edge/`, `/coordinator/`).
- `/rust/`: Directorio principal del prototipo distribuido que alberga los diferentes *crates* o microservicios.
- `/netem/`: Infraestructura preparada para los scripts de emulación de red necesarios para evaluar el comportamiento del sistema bajo distintos escenarios de falla.
- `/docs/`: Directorios establecidos para albergar los futuros manifiestos de despliegue y diagramas técnicos avanzados.

Archivo README: Se generó un `README.md` inicial que documenta la arquitectura base, las tecnologías empleadas y provee una guía rápida (Quickstart) para ejecutar los microservicios, cumpliendo con la especificación del primer avance.

3. Políticas de Seguridad y Gitignore

Para prevenir la exposición accidental de información sensible y artefactos pesados en el control de versiones, se implementó un archivo `.gitignore` robusto con las siguientes reglas restrictivas:

- Ignorar el directorio `target/` que contiene los binarios y artefactos de compilación pesados de Rust.
- Excluir archivos de certificados y llaves de cifrado asimétrico (`*.key`, `*.pem`, `*.p12`).
- Bloquear la subida del archivo real de configuración VPN (`wg0.conf`), permitiendo solo las plantillas ubicadas en la carpeta `/vpn/`.
- Ignorar los archivos `.env` que contengan las variables de entorno, puertos e IPs dinámicas locales.

4. Prototipo en Rust: Arquitectura de Microservicios

El sistema se divide en tres nodos lógicos con tareas claramente delimitadas. Se estructuraron los mensajes de red (`SensorReading`, `EdgeReport`, `CoordStatus`, `Heartbeat`) utilizando el *crate* `serde` para garantizar una serialización y deserialización eficiente en formato JSON.

4.1. Microservicio Sensor (`/rust/sensor/`)

El nodo sensor es responsable de capturar las lecturas de entorno. Se ha refactorizado la lógica principal para eliminar por completo el uso de IPs estáticas (hardcodeadas), adhiriéndose a las normativas de arquitectura distribuida.

- A través del módulo `std::env::var`, el sensor adquiere dinámicamente su identidad (`SENSOR_ID`) y las rutas de red a las que debe reportar (`EDGE_URL` y `COORD_HB_URL`).
- Cuenta con una lógica de *fallback* automático hacia `localhost` en caso de ejecutarse fuera del entorno Dockerizado para no romper la experiencia en pruebas locales.

4.2. Microservicio Edge (`/rust/edge/`)

Actúa como un nodo de procesamiento perimetral, recibiendo un alto volumen de datos del sensor para procesarlos y emitir resúmenes al coordinador.

- **Procesamiento Matemático:** Está programado para calcular el promedio móvil de las lecturas extrayendo porciones en **ventanas estrictas de 10 muestras**. Se implementó un control mediante `Mutex` para asegurar que, si llegan ráfagas irregulares (ej. 25 lecturas), los datos sobrantes se preserven en memoria y no se pierdan al cerrar el ciclo de evaluación.
- **Tolerancia a Fallos:** Posee un sistema de resiliencia de red que efectúa una reconexión automática hacia el nodo central aplicando un algoritmo de *backoff exponencial* cuando el envío del reporte falla. Las variables del nodo (como `EDGE_ID` y `COORDINATOR_URL`) también son inyectadas mediante el entorno.

4.3. Microservicio Coordinator (`/rust/coordinator/`)

Es el núcleo lógico que recibe los informes procesados del nodo Edge y los pulsos de vida (Heartbeats) directos del Sensor.

4.4. Evidencia de Ejecución

```
[+] up 4/4atch    d Detach
 ✔Network docker_iot-network Created
 ✔Container coordinator      Created
 ✔Container edge-processor   Created
 ✔Container iot-sensor       Created
Attaching to coordinator, edge-processor, iot-sensor
coordinator   | Iniciando nodo Coordinator...
coordinator   | Coordinador iniciado y escuchando en el puerto 3000
coordinator   | Esperando reportes y heartbeats de los nodos Edge...
edge-processor | Iniciando nodo Edge...
edge-processor | Nodo Edge 'edge-node-1' inicializado y escuchando en el puerto 4000
edge-processor | ❤️ Heartbeat configurado hacia http://coordinator:3000/heartbeat
coordinator    | [REGISTRO] Nuevo nodo edge-node-1 detectado vía heartbeat.
iot-sensor     | Iniciando nodo Sensor...
iot-sensor     | 🌡️ Iniciando sensor-temp-1...
iot-sensor     | 📡 Enviando ráfagas de datos a http://edge:4000/reading
iot-sensor     | ❤️ Heartbeat configurado hacia http://coordinator:3000/heartbeat
iot-sensor     | [Sensor] Dato enviado: 27.62°C
coordinator    | [REGISTRO] Nuevo nodo sensor-temp-1 detectado vía heartbeat.
```

Fig. A Secuencia de Arranque y Auto-registro

Se ilustra el inicio coordinado del clúster. El contenedor Coordinador levanta su servidor asíncrono y, casi de inmediato, los nodos Edge y Sensor inician sus transmisiones. Se evidencia el registro dinámico en el sistema centralizado mediante la recepción exitosa de los primeros heartbeats.

```
coordinator    | Reporte normal procesado desde edge-node-1 (10 muestras)
iot-sensor     | [Sensor] Dato enviado: 26.94°C
iot-sensor     | [Sensor] Dato enviado: 21.77°C
iot-sensor     | [Sensor] Dato enviado: 27.97°C
iot-sensor     | [Sensor] Dato enviado: 22.44°C
iot-sensor     | [Sensor] Dato enviado: 20.71°C
iot-sensor     | [Sensor] Dato enviado: 26.65°C
iot-sensor     | [Sensor] Dato enviado: 26.43°C
iot-sensor     | [Sensor] Dato enviado: 22.29°C
edge-processor | [Edge] Ventana procesada: promedio 31.06, muestras: 10
edge-processor | [Edge] Procesadas 10 lecturas. Latencia prom: 3178ms. Enviando...
edge-processor | [Edge] Reporte enviado exitosamente (intento 1)

coordinator    | Reporte normal procesado desde edge-node-1 (10 muestras)
iot-sensor     | [Sensor] Dato enviado: 29.95°C
iot-sensor     | [Sensor] Dato enviado: 26.22°C
iot-sensor     | [Sensor] Dato enviado: 27.45°C
iot-sensor     | [Sensor] Dato enviado: 29.95°C
iot-sensor     | [Sensor] Dato enviado: 27.70°C
iot-sensor     | [Sensor] Anomalía inyectada: 85.99°C
iot-sensor     | [Sensor] Anomalía inyectada: 85.12°C
iot-sensor     | [Sensor] Dato enviado: 23.08°C
iot-sensor     | [Sensor] Dato enviado: 23.54°C
iot-sensor     | [Sensor] Dato enviado: 21.01°C
iot-sensor     | [Sensor] Dato enviado: 26.97°C
iot-sensor     | [Sensor] Dato enviado: 28.70°C
edge-processor | [Edge] Ventana procesada: promedio 38.13, muestras: 10
edge-processor | [Edge] Procesadas 10 lecturas. Latencia prom: 4178ms. Enviando...
edge-processor | [Edge] Reporte enviado exitosamente (intento 1)
```

Fig. B Procesamiento End-to-End y Detección de Anomalías

Demostración del flujo de telemetría en tiempo real. El Sensor inyecta lecturas aleatorias, incluyendo simulaciones de anomalías térmicas (superiores a los 80°C). Simultáneamente,

el nodo Edge agrupa las lecturas en ventanas estrictas de 10 muestras, calcula los promedios móviles y transmite exitosamente los reportes resumidos al Coordinador.

```
edge-processor exited with code 137
iot-sensor       | [Sensor] Error al enviar al Edge: error sending request for url (http://edge:4000/reading):
coordinator      | [ALERTA] Falla detectada en edge-node-1. Sin señal por 10566 ms.
iot-sensor       | [Sensor] Error al enviar al Edge: error sending request for url (http://edge:4000/reading):
iot-sensor       | [Sensor] Error al enviar al Edge: error sending request for url (http://edge:4000/reading):
iot-sensor       | [Sensor] Error al enviar al Edge: error sending request for url (http://edge:4000/reading):
iot-sensor exited with code 137
coordinator      | [ALERTA] Falla detectada en sensor-temp-1. Sin señal por 10638 ms.
coordinator      | [RECUPERACIÓN] Nodo edge-node-1 volvió vía heartbeat.
coordinator      | [RECUPERACIÓN] Nodo sensor-temp-1 volvió vía heartbeat.
```

Fig. C Tolerancia a Fallos (Detección y Recuperación)

Prueba de caos controlada simulando la caída de un nodo. Al forzar la terminación del contenedor Edge (`exit code 137`), el hilo *watchdog* del Coordinador detecta la ausencia de latidos y emite una alerta tras superar el umbral de los 10 segundos. Posteriormente, se observa la reincorporación y recuperación automática del nodo en el clúster sin necesidad de intervención manual.

```
edge-processor   | [Edge] ❤️ Heartbeat fallido: error sending request for url (http://coordinator:3000/
heartbeat): error trying to connect: dns error: failed to lookup address information: Try again
edge-processor   | [Edge] Fallo al contactar al coordinador (intento 5): error sending request for url
(http://coordinator:3000/submit_report): error trying to connect: dns error: failed to lookup address
information: Try again
edge-processor   | [Edge] No se pudo enviar el reporte después de 5 intentos
edge-processor   | [Edge] Ventana procesada: promedio 24.82, muestras: 10
edge-processor   | [Edge] Ventana procesada: promedio 31.62, muestras: 10
edge-processor   | [Edge] Ventana procesada: promedio 25.59, muestras: 10
edge-processor   | [Edge] Ventana procesada: promedio 39.68, muestras: 10
edge-processor   | [Edge] Procesadas 40 lecturas. Latencia prom: 17473ms. Enviando...
edge-processor   | [Edge] ❤️ Heartbeat fallido: error sending request for url (http://coordinator:3000/
heartbeat): error trying to connect: dns error: failed to lookup address information: Try again
edge-processor   | [Edge] Reintentando en 100ms...
```

Fig. D Resiliencia de Red y Algoritmo de Backoff

Comportamiento del sistema ante fallos de conexión. Tras perder comunicación con el Hub central, los *timeouts* del cliente HTTP previenen el bloqueo del hilo principal. El nodo Edge captura el error y activa de inmediato su mecanismo de reintentos aplicando un *backoff* exponencial, garantizando que no se sature la red durante los periodos de inestabilidad.

```
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | [HEARTBEAT] Recibido de edge-axel - Latencia: 264ms
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 80ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 13ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | Reporte normal procesado desde edge-axel (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-axel (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 53ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 54ms
coordinator  | [HEARTBEAT] Recibido de edge-axel - Latencia: 675ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | Reporte normal procesado desde edge-axel (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | [HEARTBEAT] Recibido de edge-axel - Latencia: 309ms
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 124ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 25ms
coordinator  | Reporte normal procesado desde edge-axel (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-axel - Latencia: 99ms
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 0ms
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 48ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
```

Fig. Evidencia de comunicación en el sistema (sin tc netem)

Se muestra el comportamiento del sistema en condiciones normales de red, sin la aplicación de scripts de emulación de latencia. En esta captura se observa la comunicación estable entre el Edge y el Coordinator, con tiempos de respuesta consistentes y recepción correcta de heartbeats y reportes.

```
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 1749ms
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 0ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 1840ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 0ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 2611ms
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 1ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 0ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 2068ms
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 0ms
coordinator  | [HEARTBEAT] Recibido de edge-yael - Latencia: 87ms
coordinator  | Reporte normal procesado desde edge-gaboLectric (10 muestras)
coordinator  | Reporte normal procesado desde edge-yael (10 muestras)
coordinator  | Reporte normal procesado desde edge-rogelio (10 muestras)
coordinator  | [HEARTBEAT] Recibido de edge-gaboLectric - Latencia: 2152ms
coordinator  | [HEARTBEAT] Recibido de edge-rogelio - Latencia: 0ms
```

Fig. F Prueba de Red con tc netem (latencia_vpn.sh)

Se observa claramente el impacto en la comunicación entre Edge y Coordinator, comparando el comportamiento del sistema bajo condiciones de red degradadas frente a condiciones normales. Esta prueba permite validar la tolerancia a fallos y la resiliencia del pipeline distribuido ante escenarios de alta latencia.

5. Orquestación e Infraestructura VPN

El entorno de ejecución abandona el uso de servicios en la nube en favor de una infraestructura robusta y local mediante máquinas virtuales levantadas con Arch Linux utilizando QEMU, MacOS con QEMU y ubuntu server y Windows utilizando WSL2.

- **Docker Compose:** Se desarrolló un archivo de orquestación `docker-compose.yml` versión 3.8 que levanta la triada de servicios de forma aislada. Los contenedores operan dentro de una red bridge interna (`iot-network`) comunicándose fluidamente mediante la inyección de sus configuraciones en formato de variables de entorno y controlando la verbosidad de los *logs* con `RUST_LOG=info`.
- **WireGuard:** Todos los nodos de desarrollo y las máquinas virtuales ejecutan una interfaz `wg0` para participar en una VPN cifrada y lograr visibilidad directa. La configuración en `/vpn/` asiste en levantar el túnel que actúa como la columna vertebral para la transmisión de las cargas serializadas.

5.1. Declaración y Justificación de Nivel de Red

Declaración: Nivel de Red 1 (Sin CGNAT, IP pública disponible).

- **Justificación Técnica:** De acuerdo con las especificaciones de infraestructura del proyecto, el equipo declara operar bajo el Nivel de Red 1. El nodo central (Hub), administrado por Rogelio , aloja el contenedor del Coordinador y cuenta con una conexión a Internet provista de una dirección IP pública directa, libre de restricciones de *Carrier-Grade NAT* (CGNAT). Esta condición favorable permitió realizar la redirección de puertos (*Port Forwarding*) en el router de su domicilio, habilitando la escucha de tráfico UDP entrante para WireGuard.
- **Impacto en la Topología y Simulación:** Gracias a esta disponibilidad, logramos implementar y mantener la topología *hub-and-spoke* nativa sin intermediarios. Como se observa en nuestro diagrama de arquitectura, los nodos perimetrales (administrados por Gabriel, axel y yael) actúan como *spokes* y establecen sus túneles WireGuard (`wg0`) directamente contra la IP pública de Rogelio formando la subred `10.0.0.0/24`.

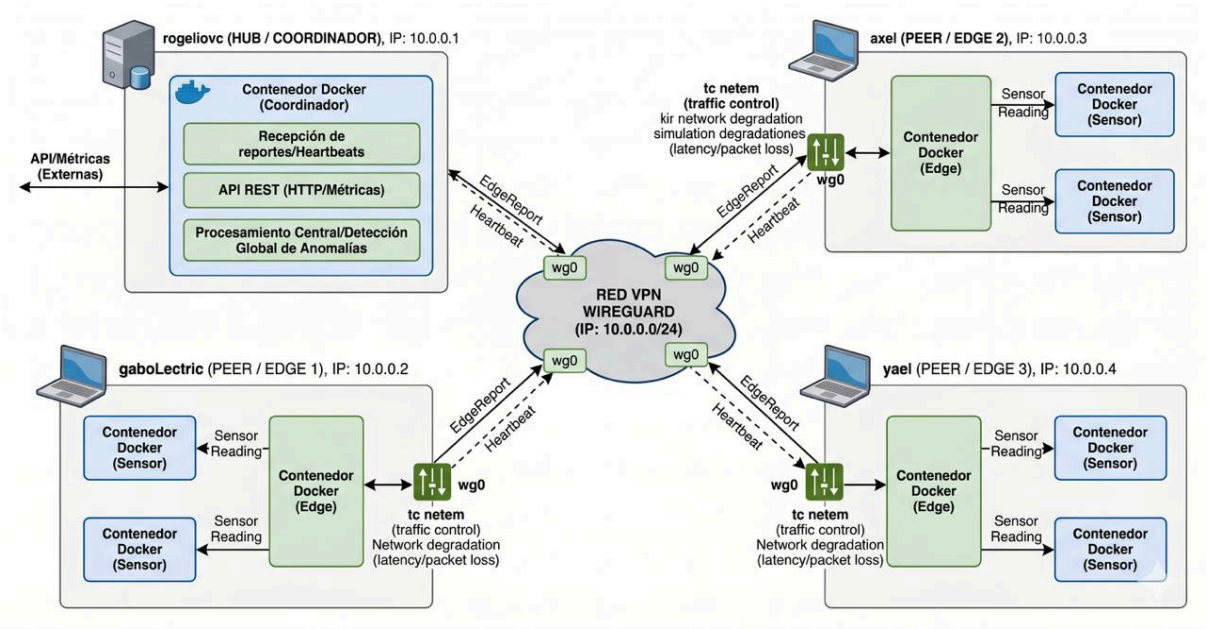

Fig. 1 Topología de red y contenedores.

Al no requerir de un VPS externo (Nivel 2) ni de servicios administrados como ZeroTier (Nivel 3), garantizamos que la latencia base (*baseline*) del enlace sea exclusivamente la latencia física entre los ISPs de los integrantes. Esto nos proporciona el entorno ideal, limpio y de menor sobrecarga posible para aislar e inyectar con precisión los escenarios de degradación de red (latencia y pérdida de paquetes) utilizando `tc netem` directamente sobre las interfaces virtuales `wg0` de cada Edge.

6. Metodología de Trabajo y Trazabilidad

Se mantiene un flujo de desarrollo riguroso, evitando commits monolíticos al final del sprint. Cada integrante participa activamente mediante un esquema de creación de ramas

(*branching*) por *feature* y reportando defectos o mejoras a través de los tableros de Issues (ej. Issue #3, #8 y #12).

- **Pull Requests:** El equipo utiliza PRs revisados para inyectar los cambios al sistema, integrando las nuevas funcionalidades, correcciones de red, y reestructuración de contenedores hacia la rama `develop`.
- **Commits Descriptivos:** Se exige el uso de mensajes de commit explicativos y granulares (ej. `"feat(sensor): refactor to use environment variables for URLs and ID"` o `"fix(edge): usar env vars y preservar lecturas sobrantes en la ventana"`), garantizando una documentación progresiva en el control de versiones.

7. Plan de Ejecución

7.1. Reunión #001

La reunión #001 del Sprint 1 (Estructura y Core del Prototipo) se llevó a cabo el **2026-04-20**. El horario de la sesión fue de **7:00 PM a 9:30 PM**. Estuvieron presentes Rogelio , Gabriel, axel y yael. **Temas y Decisiones:** Se confirmó la arquitectura del sistema de tres nodos lógicos (Sensor, Edge, Coordinator) y se decidió utilizar el formato JSON con el módulo
serde para la comunicación eficiente de mensajes de red. Se aprobó que todas las configuraciones (IDs y direcciones de red) se inyectarán exclusivamente a través de variables de entorno, evitando direcciones fijas. Finalmente, se validó la estructura de directorios inicial del repositorio, incluyendo la carpeta obligatoria `/docs/`

. **Asignación de Tareas:** Rogelio quedó encargado de configurar el *Workspace* de Cargo e integrar las dependencias base (`tokio, serde`) para el 2026-04-22. Gabriel debe implementar la lógica para calcular el promedio móvil en el nodo Edge (2026-04-24), mientras que axel finalizará la lógica de *fallback* automático del Sensor a *localhost* (2026-04-23). yael debe completar el archivo `.gitignore` robusto para el 2026-04-21.

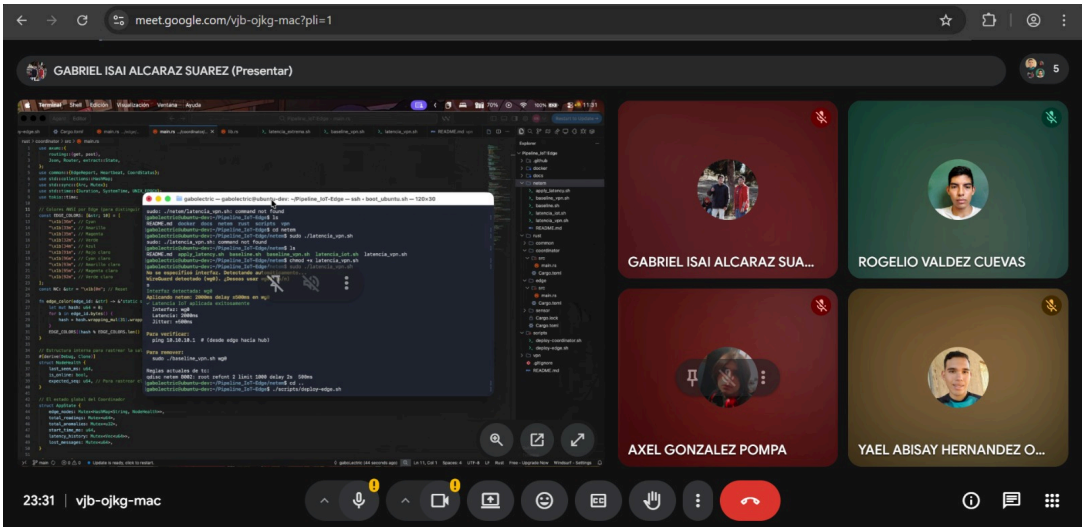

Fig. 1. Primera reunion para acordar roles y actividades

Reunión #002

La segunda reunión, Minuta #002 del Sprint 1 (Validación de Edge y Configuración de Red), se realizó el **2026-04-25** en el horario de **9:00 PM a 11:30 PM**. Estuvieron presentes los cuatro integrantes: Rogelio , Gabriel, axel y yael. **Temas y Decisiones:** Se definió que el nodo Edge procesará las lecturas en **ventanas estrictas de 10 muestras** y usará un mecanismo (`Mutex`) para guardar datos sobrantes. Para la resiliencia, se aprobó que el Edge implemente un sistema de **reintento de conexión 'inteligente'** (algoritmo de *backoff* exponencial). Rogelio confirmó que su máquina actuaría como nodo central (*Hub*) de la VPN utilizando su IP pública (Nivel de Red 1). Se planificaron las primeras pruebas de caos para validar la detección de fallos del *watchdog* del Coordinador en menos de 10 segundos. **Asignación de Tareas:** Gabriel implementará el algoritmo de *backoff* exponencial (2026-04-27). axel validará la lógica en el Edge para que no se pierdan lecturas sobrantes (2026-04-28). yael finalizará la plantilla de configuración VPN (`wg0.conf.template`) y la documentación del túnel WireGuard (2026-04-26). Rogelio creará el `docker-compose.yml` para orquestar la red interna (`iot-`) y mapear variables de entorno (2026-04-27).

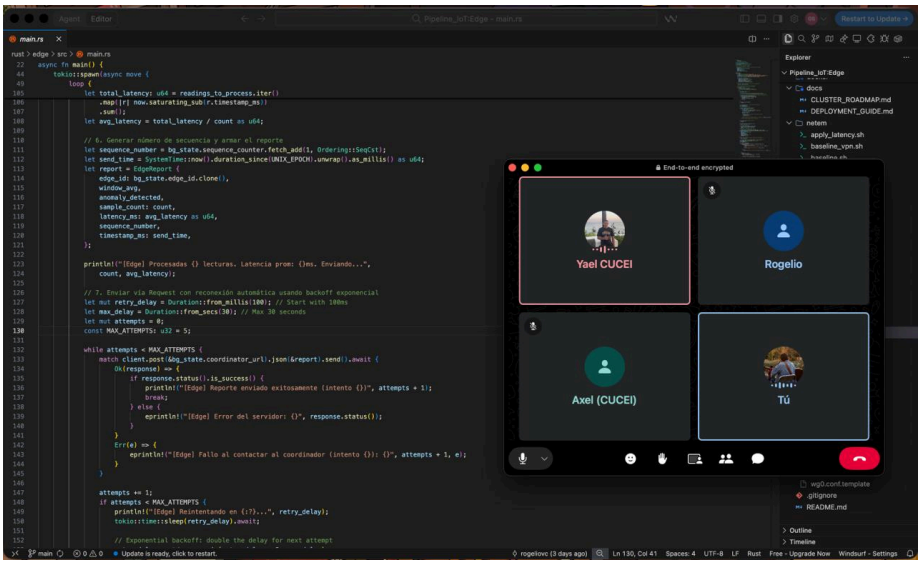

Fig. 2. Segunda reunión Validar los edge y ajustes de configuración

7.2. Componentes Faltantes por Implementar

A pesar de haber completado el núcleo del procesamiento distribuido en Rust, los siguientes componentes están pendientes para el Sprint 2: Persistencia de Datos: Actualmente el Coordinador mantiene las métricas y el historial en memoria RAM. Falta implementar una capa de almacenamiento (ej. SQLite o archivos JSON serializados) para evitar la pérdida de métricas en reinicios. Contenedor de Simulación de `Red  (NetEm)`: Si bien los Dockerfiles de los microservicios están listos, falta orquestar un servicio de "Sidecar" o scripts automatizados dentro del docker-compose.yml que apliquen reglas de tc netem de forma dinámica. Visualización de Métricas: El endpoint /status ya entrega JSON, pero falta desarrollar un dashboard minimalista (Web o CLI) que consuma estos datos para visualización en tiempo real.

7.3. Estrategia para Pruebas de Tolerancia a Fallos

La validación de resiliencia no será teórica, sino mediante caos controlado en el entorno de red: Prueba de Partición (Desconexión): Utilizaremos docker network disconnect entre el

Edge y el Coordinador para validar que el algoritmo de Backoff Exponencial del Edge reintente la conexión sin saturar el sistema. Inyección de Latencia y Pérdida: Mediante tc qdisc add dev eth0 root netem loss 20% delay 500ms, simularemos condiciones de red inestable para verificar si el Coordinador detecta las caídas en el umbral de los 10 segundos definido en el Watchdog. Recuperación Automática: Validaremos que una vez restaurada la red, el nodo sea capaz de re-registrarse automáticamente en el Coordinador mediante el envío de un nuevo Heartbeat sin intervención manual.

7.4. Metodología de Medición de Rendimiento

Las métricas se extraerán directamente del flujo de datos distribuido: Latencia E2E: Cada SensorReading lleva un timestamp de origen. El Edge calcula la diferencia al recibirlo y el Coordinador promedia estos valores para reportar percentiles P50 y P99, permitiendo identificar picos de retraso. Throughput: El Coordinador implementa un contador global de mensajes procesados dividido por el tiempo de actividad (uptime_s), permitiendo medir el rendimiento real en mensajes por segundo (msg/s). Tasa de Anomalías: Se medirá como la relación porcentual entre lecturas normales y eventos disparados (>80°C) para validar la sensibilidad del sistema.

7.5. Riesgos Técnicos Identificados Desincronización de Relojes

Al medir latencia E2E entre distintos contenedores, cualquier desfase en el reloj del sistema anfitrión afectará la precisión de los milisegundos reportados. Saturación del Coordinador (Bottleneck): El uso de Mutex bloqueantes en el Coordinador para el historial de latencias podría degradar el rendimiento si se escala a cientos de sensores simultáneos. Thundering Herd (Reconexión Masiva): En un fallo de red total, múltiples nodos intentando reconectarse simultáneamente podrían causar una denegación de servicio (DoS) involuntaria al Coordinador. Pérdida de Ventanas de Procesamiento: Dado que el Edge procesa ventanas de 10 muestras en memoria, un reinicio inesperado del contenedor resultará en la pérdida irreversible de los datos en tránsito. Conflictos de Configuración tc netem: El uso de herramientas de control de tráfico dentro de contenedores requiere privilegios de red (NET_ADMIN) que podrían comprometer la seguridad del host si no se configuran correctamente. Desbordamiento de Memoria: El historial circular para el cálculo de percentiles debe estar estrictamente limitado (actualmente a 1000 registros) para evitar que el proceso sea terminado por el OOM Killer de Docker en ejecuciones prolongadas.